

REVIEW, POSITIONING, AND DIRECT REPLACEMENT

对论文 Introduction 与 Related Work 的 严格审稿式修改方案

重点补充：学习方法是否能够绕开 LTL-to-automaton，以及为何本文方法仍然必要

项目	内容
待修改论文	On-the-Fly Temporal-Logic Task Allocation for Heterogeneous Multi-Robot Systems during Büchi Automaton Generation
参考论文	From Ambiguous Language to Verifiable Plans: Integrating Formal Synthesis and Dynamic Affordance Reasoning
重点范围	Introduction、Related Work、创新点/贡献，以及学习型 LTL 执行方法的定位
检索范围	以同行评审原始论文为主，覆盖至 2026-08-25；2026 年预印本单独标注
交付内容	审稿诊断 + 文献分类 + 完整英文替换稿 + 引文清单 + 投稿前核查表

总体判断：Introduction 需要实质性重构 (Major Revision)

论文的方法价值并未被当前引言充分表达。最强主线不是“实时重规划”，也不是泛泛的“on-the-fly”，而是在 LTL-to-automaton 构造仍在进行时，把异构团队类型/基数可行性、部分任务分配和执行状态前移到 GBA/NBA 阶段，从而避免让对当前团队无执行价值的结构继续传播，并尽早得到可执行方案。学习方法需要被正面讨论，但应被精确定位为“跨任务摊销的策略学习”或“学习技能 + 自动机高层规划”，而不是被笼统描述成已经消除了自动机与组合复杂性。

面向作者的可执行版本

包含可直接替换的英文 Introduction 与独立 Related Work

报告目录

- 1. 执行摘要与最优先修改项
- 2. 参考论文的写作模板：应模仿什么，不应复制什么
- 3. 当前 Introduction 的严格审稿诊断
- 4. 学习方法是否使本文方法失去必要性：分类、优点与边界
- 5. 推荐的 Introduction 论证结构
- 6. 可直接替换的英文 Introduction
- 7. 可直接替换的英文 Related Work
- 8. 创新点与贡献的最终表述
- 9. 建议新增文献与引用位置
- 10. 语言、术语和主张边界检查
- 11. 投稿前核查清单

1. 执行摘要与最优先修改项

本次修改的核心不是简单增加一段“学习方法的缺点”，而是重新回答审稿人最可能提出的反事实问题：如果一个经过训练的策略可以直接读取 LTL 公式并输出动作，为什么还需要构造 Büchi 自动机，更不需要在自动机构造期间做任务分配？一个有说服力的 Introduction 必须主动提出这个问题，并给出范围清楚、对学习方法公平、同时能够导向本文设计的答案。

一句话定位（建议贯穿摘要、引言和结论）

本文不是试图用另一种技巧“消除 LTL 的最坏情况复杂度”，而是针对需要一次性、可验证、与当前异构团队资源相匹配的 LTL-MRTA 实例，将团队可行性和任务分配注入自动机构造，使无执行价值的结构尽早停止传播，并在完整 NBA 尚未生成时返回可执行计划。

优先级	问题	审稿风险	建议处理
P0-1	在 Introduction 中正面讨论“显式自动机之外的学习方法”	否则审稿人会认为论文忽略了直接公式编码、LTL progression 和通用策略学习；“为什么不用学习”将成为明显缺口。	新增一段简洁定位；在 Related Work 中单设学习方法小节。
P0-2	把研究动机从“后续重规划”改为“团队可执行性进入构造过程”	当前开头和摘要容易把重规划塑造成主问题，弱化真正独特的 GBA/NBA 介入点。	第一段采用真实仓储场景，但不写具体机器人数量；由“先逻辑后资源”的流水线缺陷自然导出方法。
P0-3	区分两类学习方法	“学习绕开自动机”不是对所有工作都成立；Qiu、LTL-Transfer、DeepLTL、GenZ-LTL、cDFA、SemLTL 和 PlatoLTL 均显式使用自动机或自动机等价结构。	使用“部署时不显式构造自动机”与“自动机辅助学习”两类，而非一概而论。

优先级	问题	审稿风险	建议处理
P1-1	把方法长流程压缩为一个中心思想 + 三个设计机制	当前方法段落像 System Overview, 创新点被 VWAA/GBA/NBA/BnB 流程淹没。	先给中心句, 再用 4-5 句说明 GBA 剪枝、同步计划图、score 和 BnB。
P1-2	贡献改成有名称、可验证、范围明确的三项	“One innovation... Another distinctive aspect...” 显得试探性, 且首创性边界狭窄。	采用参考论文的“名称 + 做了什么 + 为什么重要 + 证据/范围”格式。
P1-3	收缩不稳妥的 completeness / global optimum 表述	若 GREEDYASSIGN 和节点合并仍非完备, 过强理论主张会反向损害创新可信度。	Introduction 使用 “correctness under stated assumptions” 和 “minimum over the retained plan graph”。

1.1 对“学习方法是否取代本文”的结论

- 确有工作在部署阶段不显式执行 LTL-to-automaton: 例如把公式语法树编码为神经网络, 或用 LTL progression 更新剩余公式并把它作为策略输入。它们的优势是把跨任务计算摊销到训练阶段, 部署时动作推断很快, 并能对未见公式表现出零样本泛化。
- 但这类方法并没有证明“任意长 LTL + 任意新环境 + 任意异构团队资源约束”都可被一个训练策略完整解决。其性能通常依赖训练任务分布、固定或可靠的 proposition vocabulary、低层技能/目标空间和经验性泛化。
- 覆盖 full LTL、无限时域、安全与非短视推理的较新方法, 反而经常重新使用 Büchi automata、reward machines、DFAs 或语义标注自动机。学习与自动机正在融合, 而不是形成“二选一”。
- 本文与这些方法的关键差异在问题层: 本文处理当前团队上的精确类型/基数可行性、具体机器人分配、可达性和 makespan; 学习型通用策略主要解决跨任务复用或低层控制。二者是互补关系。

依据说明: 对当前稿件的判断来自其第 1-3 页 Introduction/Related Work/Research Gap; 对参考写法的总结来自参考论文第 1-3 页; 学习文献的优缺点依据各论文原文的任务设定、方法表示与作者公开列出的限制。

2. 参考论文的写作模板: 应模仿什么, 不应复制什么

参考论文的优势不是句式华丽, 而是论证顺序稳定。它先用一个容易理解的实际失败场景建立“为什么现有系统不够”, 再将现有范式组织为清楚的对照, 随后把缺口转化为若干设计要求, 最后用一个中心思想统领模块并给出有名称的贡献。这个模板非常适合你的论文, 但需要把“语义-执行鸿沟”替换为“逻辑进展与异构资源可执行性被分离”的核心矛盾。

参考论文功能	参考论文的实现方式	对你论文的对应写法
现实入口	一个日常指令说明语义有多种物理实现	仓储/工厂任务说明一个全局任务同时包含先后、并行、持续和多机器人协作; 不列具体机器人数量。
核心矛盾	固定符号接口 vs. 开放语义	任务无关的自动机构造 vs. 当前团队的类型、基数、可达性与执行状态。

参考文献功能	参考文献的实现方式	对你论文的对应写法
范式对照	硬编码可靠但不泛化；学习灵活但缺少验证	自动机方法可验证但前处理重；直接公式学习可摊销推断但依赖训练分布和稳定符号；强 full-LTL 学习方法仍常用自动机。
具体后果	缺少替代物导致执行失败	先构造完整自动机导致团队不可执行/不必要同步的转移继续传播，延迟首次可执行分配。
设计要求	三项 requirement	语义保持的团队剪枝；分配与 NBA 同步；首次解与后续 makespan 改进兼容。
中心思想	一句 central idea 统领模块	把 LTL-to-automaton construction 视为 allocation-aware search，而非与规划隔离的预处理。
贡献	编号、命名、范围和证据	Team-Aware GBA Pruning; Joint NBA/Plan-Graph Construction; Anytime Refinement and Retained-Graph Reuse。
Related Work	按研究主题分小节，末尾单独总结 gap	自动机规划；结构化/anytime LTL-MRTA；学习型 temporal-logic execution；总结差异。

2.1 不应照搬的部分

- 不要模仿参考文献的具体技术词汇，例如 runtime contract、functional equivalence 或 hierarchical recovery；这些属于对方问题，不属于你的核心贡献。
- 不要为了“场景化”把第一段写成算例。机器人类型、数量、区域编号和具体公式应留在 Problem Description 或 Example 1。
- 不要把所有学习方法归结为“没有形式保证”。部分方法对技能组合或符号层提供保证；更准确的批评应是保证对象、训练假设和本文的异构分配对象不同。
- 不要声称本文解决了 LTL-to-automaton 的最坏情况指数复杂度。实验能支持的是早期剪枝降低传播的转移数、缩短首次可行解时间，并改善实际规模表现。

3. 当前 Introduction 的严格审稿诊断

当前版本已经具备完整技术链，但 Introduction 的叙述仍以“背景 - 方法流程 - 文献综述 - 再次复述方法”为主。审稿人能够看到算法组件，却需要自行推断这些组件为什么必须在自动机构造阶段发生。加入学习文献后，如果仍沿用当前结构，篇幅会进一步增加而主线更弱，因此需要重构而不是简单插段。

编号	问题	严格审稿人会如何理解	修改动作
I1	第一段场景仍偏泛化	仓储检查例子说明了时序和协作，但没有揭示“先构造自动机、后分配”的实际代价。	在段末明确：任务公式不指定机器人身份，当前班次团队决定可执行性；因此逻辑进展和团队资源必须共同推理。
I2	缺少学习范式	读者会问：直接输入 LTL 的通用 RL policy 是否已经避免自动机膨胀。	Introduction 在 Related Work 前增加一段；Related Work 新增独立小节。

编号	问题	严格审稿人会如何理解	修改动作
I3	方法过早进入 VWAA/GBA/NBA 细节	像方法章节摘要，中心思想被组件列表掩盖。	先写“allocation-aware automaton construction”；细节压缩为三个机制。
I4	“完整 NBA 后再规划”写得过于绝对	sampling/on-the-fly/structured methods 并非都显式完成全产品后才搜索。	改为“many top-down pipelines separate translation from allocation”并限定比较对象。
I5	真正独特的 GBA 介入点没有先被突出	读者可能把方法理解为 LTL2BA + greedy + BnB 的组合。	把“team feasibility before GBA insertion”和“allocation during GBA-to-NBA”作为贡献 1、2。
I6	Related Work 只有 bottom-up/top-down 一条分类轴	缺少经典 on-the-fly translation/model checking；缺少 formula-conditioned RL、skill transfer、automata embeddings。	改成三类主题，并在结尾逐项对照本文。
I7	Summary and Research Gap 实际是第二次方法概述	没有形成“文献共同缺口 -> 本文解决”的闭环。	Related Work 末尾只保留 1 个 gap 段；贡献放回 Introduction。
I8	首创性声明过窄且容易被关键词检索挑战	“first ... task allocation during score-guided NBA construction”看似新，但没有说明 GBA 和 execution state 的组合。	将首创性限定为“inject type/cardinality feasibility into GBA construction and perform execution-state-aware allocation during GBA-to-NBA construction”。
I9	重规划占比过高	会使读者误判论文主贡献为 failure recovery。	第一段不提故障；方法末尾和第三项贡献中将 graph reuse 写成后续优势。
I10	理论主张可能超出算法	Greedy assignment 或代表节点合并若非完备，soundness/completeness/global optimum 会被追问。	引言采用条件化表述；理论部分与算法真正保存的解空间保持一致。
L1	语法和搭配	“Formal languages such as LTL provides”主谓不一致；“during generation”搭配生硬。	改为“LTL provides”；标题和正文统一用“during construction”。

3.1 当前段落应如何重排

当前段落	当前功能	推荐去向
第 1 段	应用 + 仓储例子 + LTL	保留场景，但将段末改成“当前团队可执行性为何必须前移”。
第 2 段	定义 LTL-MRTA + 复杂性	保留定义，缩短一般性复杂性描述。

当前段落	当前功能	推荐去向
第 3 段	快速首次解的重要性 + 树/偏序方法	拆分：实际价值留在 Introduction；具体方法比较移到 Related Work。
第 4-5 段	完整方法流程 + 故障修复	压缩为“中心思想 + 三机制”；故障修复只留一句。
A. Related Work	bottom-up/top-down + 多类方法	移为独立 Section II，分三小节。
B. Summary and Research Gap	重复方法与首创性	删除；以编号贡献替代。

4. 学习方法是否使本文方法失去必要性

这一问题必须回答，但回答方式决定论文是否显得“了解领域”还是“回避竞争路线”。最稳妥的写法不是证明学习方法不行，而是说明它们优化的是不同的成本结构和保证对象。下面的分类是本文最需要补上的内容。

4.1 两类方法必须分开

A. 部署时不显式构造任务自动机（真正接近你的担忧）

Kuo et al. 将 LTL 语法结构编码为组合循环网络；LTL2Action 使用 LTL progression，并用 GNN/GRU/LSTM 表示剩余公式。训练完成后，策略可以在推断阶段直接根据环境状态和公式表示选择动作，而不必先为每个新公式显式生成 Büchi automaton。优势是训练成本可在大量任务上摊销，推断快，并能对未见公式零样本泛化。

B. 学习技能或策略，但仍使用自动机/等价结构做高层推理

GCRL-LTL 把新 LTL 转为 Büchi automaton 并在其图上做加权搜索；LTL-Transfer 使用 reward machine；Skill Machines、cDFA embeddings、DeepLTL、GenZ-LTL、SemLTL 和 PlatoLTL 分别使用自动机式技能组合、DFA/LDBA 表示、reach-avoid 分解或语义 LTL-to-automata translation。它们说明学习能够替代或加速低层控制，但复杂时序的记忆与组合结构仍常由自动机承担。

4.2 代表性文献的优点与相对边界

方法	推断表示	是否绕开显式自动机	主要优势	相对本文的边界
Kuo et al., IROS 2020	组合 RNN 直接编码公式语法	是（部署时）	未见公式/地图的零样本执行；推断阶段无需每个任务生成自动机	训练分布与经验性泛化；没有处理异构机器人类型、基数、可达性和 makespan 分配
LTL2Action, ICML 2021	LTL progression + GNN/GRU/LSTM	是（显式自动机）	可在大组合任务空间中泛化；LTL 预训练可提高样本效率	作者明确假设 noise-free labeling、固定 propositions，并依赖任务分布；目标是学习 policy 而非实例级 MRTA

方法	推断表示	是否绕开显式自动机	主要优势	相对本文的边界
GCRL-LTL, NeurIPS 2023	Büchi graph + goal-conditioned RL	否	无需在 LTL 任务空间训练即可执行新的 omega-regular 目标	依赖已学习的 goal-conditioned skills 和 proposition goal space; 仍先构造/搜索 Büchi automaton; 不做团队分配
LTL-Transfer, ICRA 2024	Reward machine + 可迁移 options	否	将已学技能零样本组合到新时序任务, 并保留安全约束	需要训练好的技能与 reward-machine 结构; 作者把长期规划和 optimality 列为后续方向
Skill Machines, ICLR 2024	自动机式 skill composition	否/等价结构	对正则语言任务组合技能, 并可给出可证明的满足性	要求足够的 skill primitives; 针对 regular fragments, 不是任意异构团队组合优化
cDFA Embeddings, NeurIPS 2024	组合 DFA + GNN embedding	否	将正式语义与非短视 policy conditioning 结合	论文自身指出 DFA concept class/size 带来编码与计算挑战; 处理 learned policy, 不是团队 assignment
DeepLTL, ICLR 2025	LDBA + reach-avoid sequence embedding	否	支持 full/infinite-horizon LTL、安全与非短视策略	明确利用 Büchi automata; 需要训练通用策略, 保证对象与本文的实例级资源可行性不同
GenZ-LTL, NeurIPS 2025	Büchi automaton + 单个 reach-avoid subgoal	否	对未见 LTL 公式零样本泛化; 处理嵌套长时域任务与安全约束	测试时先构造等价 Büchi automaton; 按单子目标求解可能牺牲全局最优性; 不做异构团队分配
SemLTL, IJCAI 2026	语义标注 LTL-to-automata, 按需构造状态	否	full LTL、语义 embedding、只构造所需自动机片段	最新路线同样采用 on-the-fly automata; 目标是通用策略表示, 不是当前团队的精确分配
PlatoLTL, arXiv 2026 (预印本)	参数化 proposition + LDBA/reach-avoid 表示	否	扩展到未见 proposition 实例, 削弱固定词汇表限制	仍需训练策略与参数化谓词/环境族; 不认证机器人类型、基数、可达性与 makespan

4.3 可以写入论文的核心结论

- 不要写: “Learning-based methods cannot handle long LTL formulas.” 这一说法会被 LTL2Action、DeepLTL 等工作直接反驳。
- 建议写: “Direct formula-conditioned policies amortize reasoning across a training task distribution, whereas the present work addresses training-free, instance-specific allocation for the currently available heterogeneous team.”
- 不要写: “Learning has no formal guarantees.” 建议写: “The guarantees and assumptions of learning-based methods concern policy/skill execution under a learned task interface, whereas our problem additionally requires explicit type/cardinality feasibility and robot assignment for the current instance.”

- 关键句：“Learning changes where computation is paid rather than removing the temporal-resource coupling.”这句话能够同时承认学习的价值并说明本文仍然必要。
- 把两条路线写成互补：学习可在未来作为 travel-time estimator、low-level skill policy 或 rollout heuristic；本文提供高层可验证的分配与自动机进展结构。

4.4 审稿人问题与推荐回答

审稿人可能提问	推荐回答逻辑
为什么不训练一个 universal LTL policy?	需要代表性训练任务、稳定 proposition semantics 和可复用低层技能；本文面对的是当前异构团队上的一次性组合分配，机器人身份、数量、位置和可达性直接改变可行集合。
学习是否解决了长公式?	它可以在某些分布上改善泛化和推断速度，但不能据此推出任意公式、任意新 proposition、任意资源配置都被完整覆盖；而 full-LTL 学习方法常重新使用自动机。
你的方法是否比学习更好?	不要进行跨问题的绝对优劣比较。强调本文提供训练前提更少、实例级可解释分配、显式接受结构和可计算 makespan；学习方法擅长跨任务摊销与低层执行。
为什么还要优化自动机构造?	因为在本文问题中自动机仍是任务语义与接受条件的载体；将团队信息前移可以避免任务无关构造产生的无执行价值结构，并使可执行计划更早出现。

5. 推荐的 Introduction 论证结构

建议完全采用参考论文的“现实问题 -> 范式对照 -> 设计要求 -> 中心思想 -> 贡献”节奏，但内容围绕你的算法。Introduction 正文建议 7 个功能段落，Related Work 单独作为 Section II。

段落	必须完成的功能	避免事项
P1 现实场景	仓储中运输、检查、维护和监控具有先后/并行/协作；机器人身份由当前团队决定；早获得可执行分配有运营价值。	不写 $r1/r2$ 、type-1 数量、区域编号或故障事件。
P2 问题定义	定义 LTL-MRTA；说明全局公式不预分配机器人；指出公式复杂度与分配组合共同增长。	不在此综述所有 MILP/search 文献。
P3 传统流水线缺口	完整或任务无关的 translation 先于 allocation；团队不可行和不必要同步的结构继续传播；瓶颈是 feasibility arrives too late。	避免“所有现有方法都先完整构造 NBA”的绝对化。
P4 学习路线	承认 formula-conditioned policy 的优势；区分 automaton-free inference 与 automata-assisted learning；指出训练假设和本文问题层差异。	不贬低、不声称学习不能处理长公式。
P5 三项设计要求	team-aware pruning；allocation-aware generation；anytime refinement。	不要提前列算法步骤。

段落	必须完成的功能	避免事项
P6 中心思想和高层方法	一句中心思想；随后概述 GBA 剪枝、NBA/plan graph 同步、score、BnB；repair 仅一句。	不要重复 System Overview。
P7 编号贡献	贡献命名、机制、实际意义、证明/实验范围。	不要用 “One innovation... Another distinctive aspect...”。

推荐的中心思想句

We treat LTL-to-automaton construction as an allocation-aware search process rather than task-independent preprocessing: temporal progressions are generated, screened against the available heterogeneous team, and extended with concrete assignments within a single construction workflow.

5.1 章节组织建议

首选结构是将 Related Work 移出 Introduction，仿照参考论文设置独立 Section II。这样能够在 Introduction 先完成论证和贡献，再在 Related Work 中展开学习方法，避免“方法还未讲清楚就进入长文献综述”。代价是后续章节编号整体顺延一次。若暂时不希望改编号，可把第 7 节提供的四个小节保留为 I-A 至 I-D；文字无需改变。

6. 可直接替换的英文 Introduction

使用说明：以下 [L1]-[L11] 是新增文献的临时标签，便于审阅。放入 LaTeX/BibTeX 后请用第 9 节建议的 citation keys 替换并让 BibTeX 自动重新编号。现有 [1]-[28] 沿用你稿件当前含义。

I. INTRODUCTION

Consider an automated warehouse in which mobile carriers, inspection platforms, and maintenance robots must jointly process incoming goods, verify storage conditions, service equipment, and monitor restricted areas. Some operations may proceed in parallel, others must follow a prescribed order, and some require several robots with particular capabilities to act together. The exact robots that execute these operations should be selected from the team available for the current shift rather than hard-coded in the mission description. In such a setting, a planner is useful only if it can expose an executable allocation early enough for operations to begin. A pipeline that first enumerates logically admissible task progressions and only afterward asks whether the current team can execute them may spend substantial computation on structures that have no operational value for the deployed system.

Linear temporal logic (LTL) provides a compact and mathematically precise language for expressing ordering, persistence, recurrence, safety, and synchronization requirements in multi-robot missions [4]-[6]. We study multi-robot task allocation under a global LTL specification, referred to as LTL-MRTA [7]. The formula specifies the temporal mission of the team but does not prescribe the identity of the robots assigned to each action. Consequently, the planner must jointly select an accepting temporal progression and assign heterogeneous robots subject to type, cardinality, reachability, and collaboration requirements. The resulting search becomes difficult as both the specification structure and the number of admissible robot assignments grow; MILP-based and graph-search-based formulations can therefore require substantial computation even when an initial feasible plan is sufficient [7]-[10].

Many top-down approaches separate these two sources of reasoning: the LTL specification is first translated into a Büchi automaton, after which task allocation or product-space planning is performed [9], [10], [19], [20]. This separation is convenient, but the translation is driven primarily by the logical formula and is unaware of the team that will execute it. Candidate transitions may therefore be propagated even when their simultaneous type or cardinality requirements cannot be met by the available robots. Other transitions may encode avoidable synchronization between task requirements that can be realized through a less restrictive temporal progression. When these structures are carried into the nondeterministic Büchi automaton (NBA) and the downstream allocation problem, computation and memory are spent before team feasibility is considered. The practical bottleneck is therefore not only automaton size; it is also that information determining executability enters the pipeline too late.

An important question is whether recent learning-based approaches make this LTL-to-automaton bottleneck obsolete. Formula-conditioned reinforcement-learning methods can encode the syntax of an LTL formula directly in a neural policy or update the remaining formula through LTL progression, enabling a trained agent to execute previously unseen instructions without explicitly constructing a task automaton at deployment [L1], [L2]. This paradigm amortizes reasoning across a distribution of tasks and can provide fast action selection after training. Related approaches reuse goal-conditioned or task-agnostic skills to execute novel temporal specifications [L3]-[L5]. These advantages are compelling when representative training tasks, reliable proposition labels, and a stable skill or goal space are available. They do not, however, eliminate the problem considered here. Direct formula encoders provide empirical generalization over a training regime rather than an explicit instance-level allocation for an arbitrary heterogeneous team, and methods that support full infinite-horizon LTL, safety, nonmyopic reasoning, or broader proposition generalization often reintroduce Büchi automata, reward machines, or related automaton structures [L3]-[L10]. Moreover, these methods do not directly choose concrete robot subsets under type, cardinality, reachability, and makespan constraints. Learning therefore changes where computation is paid rather than removing the temporal-resource coupling. For training-free, instance-specific planning with explicit satisfaction semantics, the automata-based pipeline remains valuable, but translation and allocation should no longer be isolated.

These observations lead to three design requirements. First, robot-team feasibility should influence automaton construction before infeasible combinations propagate to later stages. Second, temporal progression and allocation should be evaluated together, so that each generated transition is checked against a concrete partial execution state and an executable plan can be recognized without waiting for a complete NBA. Third, rapid feasibility should not require an irrevocable commitment to one greedy solution; the same representation should support continued makespan refinement after the first plan is available.

To meet these requirements, we propose an allocation-aware Büchi-automaton construction framework for LTL-MRTA. The central idea is to treat LTL-to-automaton construction as a search process coupled to the current robot team rather than as task-independent preprocessing. Following the LTL2BA translation pipeline [14], the framework screens candidate transitions during generalized Büchi automaton (GBA) construction against the available robot types and cardinalities, and removes progress-consistent decomposable transitions before degeneralization when their synchronization is unnecessary. During GBA-to-NBA construction, the NBA and a plan graph are generated synchronously. Each new NBA transition induces a subtask that is immediately evaluated using the robot assignments, predicted positions, and completion times stored in the current plan node. A residual-obligation score prioritizes feasible successors that combine a lower current cost with a smaller estimated burden for the remaining actions. An executable prefix-suffix plan is returned as soon as an accepting structure with valid assignments is reached. Construction then continues to improve the plan-graph solution, which warm-starts a branch-and-bound search over the retained plan graph. The retained graph can also be re-rooted at the confirmed execution state when a robot becomes unavailable, although this reuse is a downstream benefit rather than the primary motivation for the construction.

The main contributions are as follows:

1. **Team-Aware GBA Construction.** We inject robot-type and cardinality feasibility into the VWAA-to-GBA construction and remove progress-consistent decomposable transitions before GBA-to-NBA degeneralization. This prevents team-infeasible and avoidably synchronized task progressions from entering the downstream NBA while preserving feasible accepting behavior under the stated pruning conditions.
2. **Score-Guided Joint NBA and Plan-Graph Construction.** We construct the NBA and an execution-state-aware plan graph in one workflow. Every generated NBA transition is immediately tested through task allocation, and a residual-obligation score orders feasible successors using both accumulated makespan and estimated remaining travel. The framework can therefore expose a high-quality executable plan before the complete NBA and plan graph are available.
3. **Anytime Refinement and Reusable Planning Representation.** The best plan obtained during joint construction initializes a branch-and-bound optimizer that refines makespan over the retained plan graph. The same graph supports reassignment of unfinished tasks when the available team changes. We provide complexity and correctness analyses under the stated assumptions and evaluate first-plan latency, first-plan quality, scalability, repair time, and physical execution against representative baselines.

To the best of our knowledge, this is the first LTL-MRTA framework that injects robot-type and cardinality feasibility into GBA construction and performs execution-state-aware task allocation during GBA-to-NBA construction. This statement is intentionally limited to the combined construction mechanism rather than to on-the-fly LTL translation or learning-based temporal-logic execution in general.

The remainder of this paper is organized as follows. Section II reviews automata-based, structure-exploiting, and learning-based approaches to temporal-logic planning. Sections III and IV present the preliminaries and formulate the LTL-MRTA problem. Section V provides an overview of the proposed framework. Sections VI and VII introduce GBA-level pruning and joint NBA/plan-graph construction, respectively. Section VIII presents branch-and-bound refinement, and Section IX describes plan adaptation under robot unavailability. Sections X and XI provide complexity and theoretical analyses. Sections XII and XIII report numerical and physical experiments, and Section XIV concludes the paper.

6.1 为什么这一版更适合你的论文

- 第一段通过实际仓储业务说明“为什么需要团队信息前移”，但没有把场景写成具体算例，也没有把重点落在失效后重规划。
- 学习方法在提出本文设计要求之前出现，主动回答了“为什么不直接学一个 policy”的疑问；同时承认其优势，避免 straw-man comparison。
- 真正的创新被表述为两个介入点：GBA construction 中的团队可行性，以及 GBA-to-NBA construction 中的 execution-state-aware allocation。
- “首次解快”被解释为结构性结果，而不是营销化速度词：无执行价值的转移少传播，分配不再等待完整 NBA。
- 重规划只作为 retained graph 的复用价值出现，不再主导第一段和问题定义。

7. 可直接替换的英文 Related Work

推荐把本节设置为独立 Section II，模仿参考论文的组织：每个小节先概括路线，再列代表工作和优势，随后指出共同边界，最后用一两句说明本文差异。若不调整章节编号，可保留为 Introduction 下的 A-D 小节。

II. RELATED WORK

Our work lies at the intersection of automata-based temporal-logic planning, scalable task allocation for global LTL specifications, and learning-based execution of temporal-logic instructions. We organize the literature according

to these three perspectives because they address different sources of complexity: symbolic task progression, heterogeneous-team allocation, and the amortization of control or planning across task distributions.

A. Automata-Based Temporal-Logic Planning for Multi-Robot Systems

The classical automata-theoretic approach translates an LTL specification into a Büchi automaton and searches for an accepting run in a product with a model of the robot or environment [9], [19]. On-the-fly model checking and translation methods generate only the portions required by the search and simplify intermediate structures during construction [L11], [14]. These techniques are fundamental to the present work, but their reductions are primarily driven by formula semantics and automaton structure. They do not determine whether a candidate transition can be realized by a particular heterogeneous team or whether its task requirements admit a concrete assignment.

Multi-robot temporal-logic planning is commonly divided into bottom-up and top-down formulations. Bottom-up approaches assign local LTL specifications to individual robots and coordinate their behavior through communication, synchronization, or online reconfiguration [17], [18]. Top-down approaches begin with a global team specification and derive local plans or assignments. This paper adopts the latter setting because the robot identities associated with the required actions are not fixed in the input formula.

For global specifications, product-automaton methods provide a direct route to correct task and motion plans [9], [19], but the product grows rapidly with the robot state spaces and the automaton derived from the formula. Sampling-based methods incrementally approximate the product space and provide probabilistic completeness or asymptotic optimality under their assumptions [10], [20]. Their scalability is attractive, although many formulations encode robot identities in the propositions; enumerating admissible robot combinations in the global formula is undesirable when assignments must remain flexible. Simultaneous task-allocation-and-planning frameworks decompose a global specification and combine team models with allocation [16], [21], while MILP-based methods encode temporal and resource constraints in an optimization program [7], [8], [22]. These approaches provide principled solutions but can incur substantial preprocessing or optimization cost before an executable allocation is available, particularly for complex specifications and large teams.

Our approach differs in where team information enters the pipeline. Instead of treating the Büchi automaton as a completed task model on which allocation is subsequently solved, we use robot-type and cardinality information while the GBA is constructed and propagate concrete assignments while the NBA is generated. Thus, the automaton fragment supplied to planning is already aligned with the available team.

B. Structure-Exploiting and Anytime LTL-MRTA

A second line of work improves scalability by exploiting task structure or by constructing plans incrementally. Planning-decision-tree methods jointly represent task progression and robot assignment and can return feasible plans rapidly for large teams [11], [15], [23]. Their incremental character is closely related to the practical objective of this paper. However, when a single representative assignment is selected greedily at each branch, the resulting plan can be sensitive to local decisions and may require a separate mechanism for systematic improvement.

Partial-order and poset-based methods extract precedence and concurrency relations from the task automaton and then solve the resulting assignment and scheduling problem [3], [25]. They support anytime optimization and can expose parallelism that is hidden in a monolithic product. The extraction step, however, is itself a preprocessing stage whose cost and representation size can increase with specification complexity. Hierarchical temporal-logic formulations and decomposition-based methods further reduce complexity by requiring or exploiting conjunctive, hierarchical, or otherwise structured mission descriptions [24], [28]. These methods are effective when such structure is available, but part of the computational burden is shifted to specification design, decomposition, or the construction of intermediate partial orders.

The proposed framework is complementary to these ideas. It does not require the user to supply a hierarchy or explicit partial order. Instead, it intervenes in the standard LTL2BA pipeline: team-infeasible and progress-consistent decomposable GBA transitions are removed before NBA construction, and feasible assignments are evaluated as NBA

transitions appear. The first plan and subsequent improvement are therefore produced within one retained graph representation rather than by separate preprocessing, feasibility, and optimization stages.

C. Learning-Based Execution of Temporal-Logic Specifications

Learning-based methods offer an alternative way to reduce repeated planning effort. Kuo et al. [L1] use a compositional recurrent architecture whose structure follows the input LTL formula, enabling a trained agent to execute previously unseen formulas without explicitly constructing an automaton for each test task. LTL2Action [L2] progresses the LTL instruction online and encodes the remaining formula using recurrent or graph neural networks. These methods demonstrate that formula-conditioned policies can generalize to unseen and longer instructions, and that LTL semantics can be partially amortized through pretraining. Their deployment-time advantage is therefore real: once a suitable policy has been trained, action selection need not wait for symbolic automaton construction.

The corresponding assumptions are different from those of instance-specific LTL-MRTA. LTL2Action formulates learning over a task distribution, assumes a noise-free labeling function, and evaluates generalization over a fixed set of propositions, with unseen propositions identified as a direction requiring further study [L2]. Recent PlatoLTL [L10] directly weakens this limitation by parameterizing propositions and demonstrating zero-shot transfer to unseen proposition instances. This advance should be acknowledged rather than obscured. It nevertheless remains policy generalization within trained predicate and environment families, not an exact allocation certificate for a newly supplied heterogeneous team. More generally, direct neural encoders rely on learned environment dynamics or low-level capabilities; they do not explicitly enumerate robot subsets or certify type, cardinality, reachability, and disjointness constraints for the current team, nor do they directly optimize the makespan of a concrete multi-robot allocation.

Other learning approaches retain an automaton or an automaton-like structure at the task level. GCRL-LTL converts the test specification into a Büchi automaton and uses a learned goal-value model to weight graph search, thereby adapting goal-conditioned policies to omega-regular tasks without training over the LTL task space [L3]. LTL-Transfer composes learned options through reward machines to solve new temporal tasks safely [L4], while Skill Machines compose learned primitives for regular-language specifications [L5]. Compositional automata embeddings represent temporal goals as Boolean combinations of DFAs and condition a policy on their graph embeddings [L6]. DeepLTL explicitly exploits limit-deterministic Büchi automata to support nonmyopic reasoning, safety constraints, and infinite-horizon LTL [L7]. GenZ-LTL constructs an equivalent Büchi automaton at test time and decomposes it into reach-avoid subgoals for safe zero-shot execution of arbitrary LTL specifications [L8]. SemLTL uses semantically labelled LTL-to-automata translations and constructs only the required automaton states on the fly [L9]. PlatoLTL also derives structured reach-avoid representations from an LDBA while extending generalization across proposition instances [L10]. Collectively, these results suggest that learning and automata are increasingly integrated: learning supplies reusable control competence or task embeddings, while automata retain temporal memory and acceptance structure.

These methods are promising complements to the present framework. Learned skills or travel-time models could replace low-level motion estimates, and learned value functions could guide successor ordering. The present paper addresses a different layer: given the current heterogeneous team and a global LTL mission with unassigned actions, it constructs an explicit executable allocation and refines its makespan without requiring a task-policy training distribution. The contribution is therefore not that symbolic planning universally dominates learning, but that allocation-aware automaton construction remains necessary when exact team resources and per-instance assignments determine feasibility.

D. Summary and Research Gap

Existing work has reduced temporal-logic planning cost by sampling product spaces, decomposing specifications, extracting partial orders, incrementally constructing planning trees, or amortizing control through learned formula and automaton representations. These advances leave a specific gap for global LTL-MRTA with heterogeneous, unassigned actions. Automata-based methods generally separate task translation from concrete allocation; structure-

exploiting methods may require additional preprocessing or a restricted mission form; and learning-based methods either depend on a task/skill training regime or continue to use automata without addressing exact robot-type and cardinality allocation for the current team. To the best of our knowledge, no prior framework simultaneously screens team infeasibility during GBA construction, performs execution-state-aware allocation during GBA-to-NBA construction, and uses the resulting plan graph for both early plan extraction and subsequent makespan refinement. This is the gap addressed by the proposed method.

7.1 Related Work 中必须避免的三种不公平比较

- 不要把 GCRL-LTL、DeepLTL、GenZ-LTL、SemLTL 或 PlatoLTL 放进“绕开自动机”一类；它们明确依赖 Büchi/LDBA、reach-avoid automaton structure 或 semantic automata。
- 不要把 Kuo/LTL2Action 的限制写成“无法泛化”。它们的主要贡献正是零样本公式泛化；正确边界是泛化依赖训练设定、标签/命题接口，且目标不是异构团队精确分配。
- 不要用你的 T_{first} 与学习方法的 policy inference latency 直接比较；前者包含实例级符号构造和分配，后者把大量成本前移到训练阶段，成本核算基础不同。

8. 创新点与贡献的最终表述

贡献表述应从“流程中做了哪些步骤”提升为“改变了哪个信息进入规划的时机、因此避免了什么、证据是什么”。下面给出投稿时建议保留的三项，以及理论修正后可使用的强化版本。

贡献名称	建议正文	审稿价值
1) Team-Aware GBA Construction	Inject robot-type/cardinality feasibility before candidate GBA transitions are committed, and remove progress-consistent decomposable transitions before degeneralization.	突出最独特的中间自动机介入点；不是普通 NBA 后剪枝。
2) Score-Guided Joint NBA/Plan-Graph Construction	Generate the NBA and execution-state-aware assignments together; use residual obligations to order only feasible successors; extract a plan upon first accepting closure.	将早期可执行解与算法机制直接对应，不只是“更快”。
3) Anytime Refinement and Retained-Graph Reuse	Use the plan-graph solution as the BnB incumbent and reuse the retained graph after team changes; report first-plan latency, cost, scalability, and repair time.	把优化与修复写成同一表示的后续价值，不抢占主线。

8.1 保守且稳妥的首创性声明

推荐使用

To the best of our knowledge, this is the first LTL-MRTA framework that injects robot-type and cardinality feasibility into GBA construction and performs execution-state-aware task allocation during GBA-to-NBA construction.

这句话把首创性限定在两个机制的组合，而不是声称首次使用 on-the-fly、首次使用学习/非学习方法、或首次在 NBA 上规划。它比当前 “first ... during score-guided NBA construction” 更能体现技术实质，也更难被一篇只做 expansion ordering 的论文击中。

8.2 理论条件尚未完全修正时的推荐表述

- We establish correctness of every returned plan with respect to the LTL specification and characterize preservation under the stated GBA-pruning and plan-graph assumptions.
- The branch-and-bound stage returns the minimum-makespan plan over the assignments and paths represented in the retained plan graph.
- Numerical and physical experiments evaluate first-plan latency, first-plan cost, scalability, and repair time.

8.3 只有在算法/证明补强后才使用的版本

前提

GREEDYASSIGN 必须替换为完备可行匹配（或证明现有顺序贪心与可行性等价），并且计划图必须保留所有 nondominated execution labels（或给出单一代表状态的严格 dominance 证明）。

- We establish soundness and completeness of the joint construction: every returned plan satisfies the LTL mission, and every feasible accepting plan is represented by at least one retained nondominated label.
- The branch-and-bound stage returns the global minimum-makespan plan under the complete transition and assignment model.

9. 建议新增文献与引用位置

以下文献足以覆盖审稿人对 “直接学习公式、技能迁移、自动机嵌入、full LTL 学习、经典 on-the-fly” 的主要疑问。正文中不必全部放在 Introduction 的短段里；Introduction 引 5-7 篇即可，其余在 Related Work 展开。

标签	IEEE 风格条目	建议引用位置	建议 BibTeX key
[L1]	Y.-L. Kuo, B. Katz, and A. Barbu, “Encoding Formulas as Deep Networks: Reinforcement Learning for Zero-Shot Execution of LTL Formulas,” in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), 2020, pp. 5604-5610, doi: 10.1109/IROS45743.2020.9341325.	Introduction 学习段; Related Work C 直接公式编码。	Kuo2020Encoding
[L2]	P. Vaezipoor, A. C. Li, R. T. Icarte, and S. A. McIlraith, “LTL2Action: Generalizing LTL Instructions for Multi-Task RL,” in Proc. 38th Int. Conf. Mach. Learn. (ICML), PMLR, vol. 139, 2021, pp. 10497-10508.	Introduction 学习段; Related Work C, 优点与作者列出的 assumptions。	Vaezipoor2021LTL2Action

标签	IEEE 风格条目	建议引用位置	建议 BibTeX key
[L3]	W. Qiu, W. Mao, and H. Zhu, “Instructing Goal-Conditioned Reinforcement Learning Agents with Temporal Logic Objectives,” in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 36, 2023, pp. 39147-39175.	说明无需 LTL task-space 训练, 但仍基于 Büchi graph search。	Qiu2023GCRLTL
[L4]	J. X. Liu, A. Shah, E. Rosen, M. Jia, G. Konidaris, and S. Tellex, “Skill Transfer for Temporal Task Specification,” in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), 2024, pp. 2535-2541, doi: 10.1109/ICRA57147.2024.10611432.	技能复用/zero-shot; reward-machine 路线。	Liu2024LTLTransfer
[L5]	G. Nangue Tasse, D. Jarvis, S. James, and B. Rosman, “Skill Machines: Temporal Logic Skill Composition in Reinforcement Learning,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2024.	正则语言/skill composition; 避免把学习方法一概说成无保证。	Tasse2024SkillMachines
[L6]	B. Yalcinkaya, N. Lauffer, M. Vazquez-Chanlatte, and S. A. Seshia, “Compositional Automata Embeddings for Goal-Conditioned Reinforcement Learning,” in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 37, 2024, pp. 72933-72963.	自动机 embedding; 说明学习与自动机结合。	Yalcinkaya2024AutomataEmbeddings
[L7]	M. Jackermeier and A. Abate, “DeepLTL: Learning to Efficiently Satisfy Complex LTL Specifications for Multi-Task RL,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2025.	full/infinite-horizon LTL; 显式利用 LDBA, 直接支撑你的定位。	Jackermeier2025DeepLTL
[L8]	Z. Guo, İ. Işık, H. M. S. Ahmad, and W. Li, “One Subgoal at a Time: Zero-Shot Generalization to Arbitrary Linear Temporal Logic Requirements in Multi-Task Reinforcement Learning,” in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 38, 2025, doi: 10.52202/085713-2599.	最新 safe/full-LTL zero-shot 方法; 明确在测试时构造 Büchi automaton。	Guo2025GenZLTL
[L9]	A. Abate, G. De Giacomo, M. Jackermeier, J. Kretínský, M. Prokop, and C. Weinhuber, “Semantically Labelled Automata for Multi-Task Reinforcement Learning with LTL Instructions,” in Proc. Int. Joint Conf. Artif. Intell. (IJCAI), 2026; full version arXiv:2602.06746.	覆盖 2026 最新路线; semantic on-the-fly automata 与 universal policy 的融合。	Abate2026SemLTL

标签	IEEE 风格条目	建议引用位置	建议 BibTeX key
[L10]	J. Cloete, M. Jackermeier, I. Havoutis, and A. Abate, “PlatoLTL: Learning to Generalize Across Symbols in LTL Instructions for Multi-Task RL,” arXiv:2601.22891, 2026. (Preprint.)	公平回应 “固定 propositions” 限制正在被解决；仍基于 LDBA/reach-avoid 表示。	Cloete2026PlatoLTL
[L11]	R. Gerth, D. Peled, M. Y. Vardi, and P. Wolper, “Simple On-the-Fly Automatic Verification of Linear Temporal Logic,” in Protocol Specification, Testing and Verification XV, 1995, pp. 3-18.	Related Work A; 表明 on-the-fly translation/model checking 本身不是本文首创。	Gerth1995OnTheFly

9.1 推荐的引用分配

位置	建议引用
Introduction 学习段第 1 句（直接公式）	[L1], [L2]
Introduction 学习段第 2 类（技能复用）	[L3]-[L5]
Introduction “full LTL / broader generalization methods reintroduce automata”	[L3], [L6]-[L10]
Related Work A（on-the-fly）	[L11], [14]
Related Work C 第一段	[L1], [L2]
Related Work C 第二/三段	[L3]-[L10]

9.2 为什么这些文献已经足够

这组文献覆盖了审稿人会寻找的完整光谱：直接把公式输入网络（Kuo）；基于 progression 的通用策略（LTL2Action）；不在 LTL 任务空间训练但仍搜索 Büchi automaton（GCRL-LTL）；reward-machine/skill composition（LTL-Transfer、Skill Machines）；automata embeddings（cDFA）；full LTL + LDBA（DeepLTL）；Büchi subgoal decomposition（GenZ-LTL）；semantic on-the-fly automata（SemLTL）；跨 proposition 泛化但仍基于 LDBA 的 PlatoLTL；以及经典 on-the-fly formal methods（Gerth）。继续大量堆砌单任务 LTL-RL 文献不会显著增强论证，反而可能模糊本文与“学习未知动力学”问题的区别。

10. 语言、术语和主张边界检查

当前/风险表述	问题	推荐表述
“Formal languages such as LTL provides ...”	主谓不一致；LTL 是单数。	“Linear temporal logic (LTL) provides ...”

当前/风险表述	问题	推荐表述
“automaton generation”	领域内 construction 更自然；generation 容易与生成式模型混淆。	“automaton construction” / “GBA-to-NBA construction”
“existing methods typically construct the complete automaton”	对 sampling、on-the-fly、结构化方法过度概括。	“many top-down pipelines separate automaton translation from task allocation”
“infeasible and redundant transitions”	过泛，未说明相对于什么不可行/冗余。	“team-infeasible transitions and progress-consistent decomposable transitions”
“higher-quality plan”	不可测量。	“a lower-makespan plan” 或 “empirically lower first-plan cost”
“rapid” / “large-scale”	营销化，若无指标会被质疑。	使用 T_first、team size、formula/task complexity、transition count 等指标。
“toward optimality”	若优化只在 retained plan graph 上成立，范围不清。	“refines makespan over the retained plan graph”
“learning methods bypass automata”	只对部分部署表示成立。	“some formula-conditioned methods avoid explicit task-automaton construction at deployment”
“learning lacks formal guarantees”	不公平且可被反例击中。	“their guarantees and assumptions concern learned policies/skills rather than exact instance-level heterogeneous allocation”
“first ... on-the-fly”	经典 on-the-fly LTL 已久。	把 first claim 限定到 team-aware GBA + execution-state allocation 的组合。

10.1 主张必须与实验对应

Introduction 主张	正文中需要的证据
GBA-level pruning prevents unnecessary propagation	报告原始/剪枝 GBA/NBA transition counts；消融与 Case II 对照。
Allocation during construction reduces first-plan latency	Case I（等完成 NBA 后再分配）消融；统一硬件、实现和停止条件。
Residual-obligation score improves first-plan quality	discovery order vs. score-guided order；报告 J_first、T_first 和统计。
BnB refines the plan	初始 incumbent、改进曲线、最终 retained-graph optimum 或 best found。
Retained graph enables repair	故障发生时点、剩余任务、first repaired plan time、最终 repaired cost。
Learning methods do not replace this problem	不需要实验对比；通过问题设定和文献范围论证，不做不公平 runtime 比较。

10.2 需要同步修正的理论表述

Introduction 虽然不是证明章节，但过强主张会使审稿人立即检查 Algorithm 3 和 plan-node merging。若 GREEDYASSIGN 按 action index 顺序贪心选择机器人，它的失败未必意味着该 subtask 无可匹配；若同一 NBA state/subtask 只保留当前 makespan 最小的代表执行状态，较小当前 makespan 也未必支配另一组机器人位置和可用时间。因此，在算法未补强前，以下位置应同步收缩：Abstract、Introduction contribution 3、Theorem 1、Table III/IV 的 “Global Optimum”、Conclusion。

与本文主线无关但必须修正的形式语义错误

Problem Description 的 Example 1 若要表达 “始终避开 restricted region l8”，公式应使用 $\Box \neg p8$ ，而不是仅写 $\neg p8$ 。后者在标准 LTL 中只约束当前时刻。这个错误会直接削弱审稿人对 formal correctness 的信任。

11. 投稿前核查清单

- [] 第一段使用真实场景，但不出现具体机器人数量、类型编号、区域编号或失败事件。
- [] 第一段末尾明确指出：任务无关 translation 与当前团队 feasibility 分离是实际瓶颈。
- [] 在 Related Work 之前已经用一段正面讨论 formula-conditioned learning 的优点与边界。
- [] Related Work 单设 Learning-Based Execution of Temporal-Logic Specifications 小节。
- [] 把直接公式编码与 automata-assisted learning 分开，不把 GCRL-LTL/DeepLTL/GenZ-LTL/SemLTL/PlatoLTL 误写为绕开自动机。
- [] 加入经典 on-the-fly formal methods 文献，避免把 on-the-fly 本身当成首创。
- [] 方法段先给 central idea，再写三个机制；不复述完整 VWAA-GBA-NBA-BnB 流程。
- [] 贡献采用命名式三项，并把重规划降为 retained graph 的次级价值。
- [] 首创性声明只覆盖 team-aware GBA construction + execution-state-aware allocation during GBA-to-NBA。
- [] 不声称消除 LTL 最坏情况指数复杂度；只声称减少传播结构和缩短 T_{first} 。
- [] “higher-quality” “rapid” “large-scale” 均由 J_{first} 、 T_{first} 、transition count、team size 等指标替代或支撑。
- [] soundness/completeness/optimality 与 GREEDYASSIGN、node merging 和 retained plan graph 的真实范围一致。
- [] Example 1 的 always 语义已修正为 $\Box \neg p8$ 或文字改为当前时刻约束。
- [] 清理 IEEE template 作者、页眉、copyright、manuscript date 和 ACKNOWLEDGMENT 占位文本。
- [] 所有新增引用由 BibTeX 自动重排，不保留 [L1]-[L11] 临时标签。

11.1 建议的修改顺序

4. 先确定理论主张范围：是否补强 exact assignment 与 nondominated labels；这决定 Introduction 能否使用 completeness/global optimum。
5. 按第 6 节替换 Introduction，并把 Related Work 移为独立 Section II。
6. 加入第 9 节文献，使用 BibTeX key 重新编译并检查引用顺序。
7. 按第 10 节统一术语，特别是 construction、team-infeasible、retained plan graph 和 makespan。

8. 最后检查摘要、实验结论和 Conclusion 是否与新定位完全一致。

最终审稿判断

采用本报告的结构后，论文的 Introduction 将从“方法组件说明”转为一个完整的必要性论证：实际任务要求当前团队尽早获得可执行分配；传统任务无关 translation 让可行性进入太晚；直接学习可以摊销跨任务推断但不解决当前异构团队上的精确资源分配，而强 full-LTL 学习方法仍频繁使用自动机；因此需要 allocation-aware automaton construction。这个论证能够突出你的真正创新，同时避免与学习文献形成不准确的对立。

附录 A. 本报告使用的学习文献定位依据（简表）

文献	原文可支持的事实	在本稿中的使用方式
Kuo et al. 2020	组合循环网络以 LTL 公式为输入；未见公式和地图上零样本执行。	支持“部署时可不显式构造每任务自动机”的优点。
LTL2Action 2021	LTL progression + formula encoder；作者列出 task distribution、noise-free labeling、fixed propositions 假设/限制。	作为最重要的直接学习对手；优缺点都需准确写。
Qiu et al. 2023	无需 LTL task-space training；把 LTL 转为 Büchi automaton 并做 weighted graph search。	证明“学习方法”不等于“无自动机”。
LTL-Transfer 2024	通过 reward machine 和已学 options 进行 zero-shot skill transfer。	强调学习擅长技能复用，但表示和目标与 MRTA 不同。
Skill Machines 2024	对 regular-language tasks 组合 skill primitives，并提供满足性能力。	避免不公平地写“学习都无保证”。
cDFA Embeddings 2024	以 compositional DFAs 作为 policy goal representation；论文讨论 DFA size/concept class 挑战。	说明自动机 embedding 是主流融合方向。
DeepLTL 2025	明确利用 Büchi automata 处理 full/infinite-horizon、安全、nonmyopic LTL。	最强反证：full-LTL 学习没有使 automata 失去意义。
GenZ-LTL 2025	测试时构造 Büchi automaton，并逐个求解 reach-avoid subgoals；覆盖长时域与安全。	证明最新 zero-shot LTL 学习仍依赖自动机结构。
SemLTL, IJCAI 2026	语义 LTL-to-automata，自动机状态按需构造；支持 full LTL。	最新正式发表引用；说明 on-the-fly automata 与 generalist RL 正在融合。
PlatoLTL 2026 preprint	将 proposition 参数化，支持对未见 symbols 的迁移；表示仍从 LDBA/reach-avoid 结构得到。	公平承认 fixed-vocabulary 限制正在缓解，同时保持问题层区分。